GEONOVA

MASTER PLATFORM BLUEPRINT

Complete Agent Build Specification | Version 1.0

REFERENCE	STACK
N.N. Basak — Surveying and Levelling Malda Polytechnic, West Bengal	**TypeScript | Next.js | Leaflet** Supabase (PostgreSQL + PostGIS)

This document is the single authoritative specification for the GeoNova agent. Every decision — architecture, database schema, API contract, calculation standards, UI behaviour, and output format — is defined here. The agent must not deviate from these specifications.

Table of Contents

1. **Platform Vision & Principles**
2. System Architecture
3. Technology Stack
4. Database Schema
5. Survey Calculation Engine (Math Spec)
6. API Contract
7. Frontend & UI Specification
8. Agent Calculation Behaviour
9. Field Data Import
10. Output & Report Standards
11. Build Phases & Milestones
12. Error Detection Rules
13. Glossary

1. PLATFORM VISION & PRINCIPLES

1.1 What GeoNova Is

GeoNova is a professional-grade, web-based surveying calculation platform. It replaces desktop software (Trimble Business Center, Leica Infinity, Civil3D Survey tools) for routine and intermediate survey work — running entirely in the browser with no installation required.

1.2 The Three Non-Negotiables

Principle	Requirement
Calculation accuracy	A 1 cm cadastral error can void a title deed. A 0.1 m road alignment error causes structural failure. All algorithms follow Basak standards and are verified against known test cases.
Full working shown	Every output must show: formula used, substituted values, computed result, arithmetic check. Surveyors must be able to audit every step.
3-click rule	Any calculation reachable in 3 clicks from the dashboard. No deep menus.

1.3 Two Modes

Mode	Purpose
Quick Tools Mode	Standalone calculators. No project setup. Enter numbers, get results instantly. For field use and single calculations.
Project Mode	Full workflow: create project, store points, run traverse, draw parcels, generate report. For complete survey jobs.

1.4 Target Users

- Licensed land surveyors (cadastral, topographic, engineering)
- Civil engineering technicians
- Survey students (pedagogical worked examples)
- GIS professionals needing coordinate conversion tools

2. SYSTEM ARCHITECTURE

2.1 Layer Overview

The system has four layers. Each layer has a single responsibility and communicates only with adjacent layers.

Layer	Responsibility	Technology
Frontend	Interactive UI: map, calculators, tables, reports	`Next.js 14 + React + Tailwind + Leaflet`
Calculation Engine	All surveying math — pure TypeScript, no I/O, no side effects	`TypeScript modules (runs client-side)`
API + Backend	Auth, project storage, file parsing, report generation	`Next.js API Routes (Phase 1), FastAPI (Phase 3+)`
Database	Spatial storage: points, projects, traverses, parcels	`Supabase (PostgreSQL + PostGIS)`

2.2 Repository Structure

Root	`geonova/`
Engine	`geonova/engine/  ← pure TypeScript math, no dependencies`
Frontend	`geonova/app/  ← Next.js app router pages and components`
API	`geonova/app/api/  ← Next.js API routes`
DB	`geonova/supabase/  ← migrations and seed data`
Tests	`geonova/engine/__tests__/  ← 70+ passing unit tests`

2.3 Data Flow

```
User Input  →  React Component  →  engine function (client-side)  →  Display Result
User Input  →  React Component  →  /api/route  →  Supabase  →  Response
CSV Upload  →  parser.ts  →  validatePoints()  →  Supabase  →  Leaflet map
```

3. TECHNOLOGY STACK

Technology	Role
Next.js 14 (App Router)	Framework. Server components, API routes, file-based routing.
TypeScript	All code. Strict mode. No implicit any.
Tailwind CSS	Styling. Dark industrial theme. No CSS modules.
Leaflet + react-leaflet	Map rendering. Lightweight. Large plugin ecosystem.
TanStack Table	Coordinate and traverse data tables. Inline editing.
Supabase	Hosted PostgreSQL + PostGIS + Auth + Storage. No self-hosting.
shadcn/ui	Base UI components. Customised to match dark theme.
Zod	Input validation on all API routes and form inputs.
jsPDF + autoTable	Client-side PDF report generation.
Papa Parse	CSV parsing in the browser. No backend needed for file import.

! *DO NOT use Python, FastAPI, or Shapely for Phase 1-2. All math runs in TypeScript in the browser. The engine modules (types.ts, distance.ts, traverse.ts, leveling.ts, area.ts, coordinates.ts, cogo.ts, geometry.ts, curves.ts, parser.ts) are already written and tested. Use them directly.*

4. DATABASE SCHEMA

4.1 Core Tables

All tables use UUID primary keys. All timestamps are UTC. All coordinate columns use NUMERIC(14,4) to preserve 0.1 mm precision.

users

Column	Type	Notes
id	UUID PRIMARY KEY	Auto-generated
email	TEXT UNIQUE NOT NULL	Login identity
full_name	TEXT	Display name
role	TEXT DEFAULT 'surveyor'	surveyor | admin
created_at	TIMESTAMPTZ DEFAULT NOW()	

projects

Column	Type	Notes
id	UUID PRIMARY KEY	
user_id	UUID REFERENCES users(id)	Owner
name	TEXT NOT NULL	e.g. 'Kwale Subdivision 2026'
location	TEXT	Human-readable location
utm_zone	INTEGER	e.g. 37
hemisphere	CHAR(1) DEFAULT 'S'	N or S
datum	TEXT DEFAULT 'WGS84'	
created_at	TIMESTAMPTZ DEFAULT NOW()	

survey_points

Column	Type	Notes
id	UUID PRIMARY KEY	
project_id	UUID REFERENCES projects(id)	
name	TEXT NOT NULL	e.g. P1, T3, BM1
easting	NUMERIC(14,4) NOT NULL	Grid E, metres
northing	NUMERIC(14,4) NOT NULL	Grid N, metres
elevation	NUMERIC(10,4) DEFAULT 0	Reduced Level, metres
code	TEXT	Feature code
geom	GEOMETRY(POINT, 4326)	PostGIS geometry
created_at	TIMESTAMPTZ DEFAULT NOW()	

traverses

Column	Type	Notes
id	UUID PRIMARY KEY	
project_id	UUID REFERENCES projects(id)	
name	TEXT NOT NULL	
method	TEXT	bowditch | transit
closing_error_e	NUMERIC(12,6)	metres
closing_error_n	NUMERIC(12,6)	metres
linear_error	NUMERIC(12,6)	metres
precision_ratio	NUMERIC(12,1)	1 : N
precision_grade	TEXT	excellent|good|acceptable|poor
created_at	TIMESTAMPTZ DEFAULT NOW()	

traverse_legs

Column	Type	Notes
id	UUID PRIMARY KEY	
traverse_id	UUID REFERENCES traverses(id)	
sequence	INTEGER NOT NULL	Leg order
from_point_id	UUID REFERENCES survey_points(id)	
to_point_id	UUID REFERENCES survey_points(id)	
distance	NUMERIC(12,4)	metres
bearing	NUMERIC(10,6)	Decimal degrees, WCB
raw_delta_e	NUMERIC(12,4)	
raw_delta_n	NUMERIC(12,4)	
correction_e	NUMERIC(12,6)	Bowditch/Transit
correction_n	NUMERIC(12,6)	
adj_delta_e	NUMERIC(12,4)	After adjustment
adj_delta_n	NUMERIC(12,4)	

leveling_runs

Column	Type	Notes
id	UUID PRIMARY KEY	
project_id	UUID REFERENCES projects(id)	
name	TEXT NOT NULL	
method	TEXT	rise_and_fall | height_of_collimation
opening_rl	NUMERIC(10,4) NOT NULL	metres

closing_rl_known	NUMERIC(10,4)	NULL if open-ended
misclosure	NUMERIC(10,6)	metres
arithmetic_check	BOOLEAN	

leveling_readings

Column	Type	Notes
id	UUID PRIMARY KEY	
run_id	UUID REFERENCES leveling_runs(id)	
sequence	INTEGER NOT NULL	
station	TEXT NOT NULL	
staff	NUMERIC(8,4) NOT NULL	metres
type	TEXT NOT NULL	BS | IS | FS
rise	NUMERIC(8,4)	computed
fall	NUMERIC(8,4)	computed
reduced_level	NUMERIC(10,4)	computed
adjusted_rl	NUMERIC(10,4)	after correction

parcels

Column	Type	Notes
id	UUID PRIMARY KEY	
project_id	UUID REFERENCES projects(id)	
name	TEXT NOT NULL	
area_sqm	NUMERIC(16,4)	Square metres
area_ha	NUMERIC(12,6)	Hectares
perimeter_m	NUMERIC(12,4)	metres
geom	GEOMETRY(POLYGON, 4326)	PostGIS polygon
created_at	TIMESTAMPTZ DEFAULT NOW()	

calculations (audit log)

Column	Type	Notes
id	UUID PRIMARY KEY	
user_id	UUID REFERENCES users(id)	
project_id	UUID	nullable
type	TEXT NOT NULL	distance | bearing | traverse | leveling | area | curve | cogo
input_json	JSONB NOT NULL	Raw inputs
result_json	JSONB NOT NULL	Full result

created_at	TIMESTAMPTZ DEFAULT NOW()	

uploaded_files

Column	Type	Notes
id	UUID PRIMARY KEY	
project_id	UUID REFERENCES projects(id)	
file_name	TEXT NOT NULL	
file_type	TEXT	csv | txt | dat
point_count	INTEGER	Points imported
warnings	TEXT[]	Array of parse warnings
storage_path	TEXT	Supabase Storage path
uploaded_at	TIMESTAMPTZ DEFAULT NOW()	

5. SURVEY CALCULATION ENGINE — MATH SPECIFICATION

The engine is a pure TypeScript library. All functions take typed inputs and return SurveyResult<T> — either { ok: true, value: T } or { ok: false, error: string }. No function throws. No function has side effects.

> **!** *The engine is already built and tested (70 passing tests). Do not rewrite it. Import from engine/index.ts.*

5.1 Module Map

Module	Exports
types.ts	Point2D, Point3D, NamedPoint2D, LatLon, UTMCoord, DMS, SurveyResult<T>
angles.ts	toRadians, toDegrees, normalizeBearing, decimalToDMS, dmsToDecimal, bearingToString, parseDMSString
distance.ts	distanceBearing(), slopeDistance(), polarPoint()
traverse.ts	bowditchAdjustment(), transitAdjustment()
leveling.ts	riseAndFall(), heightOfCollimation()
area.ts	coordinateArea(), subdividePolygon()
coordinates.ts	geographicToUTM(), utmToGeographic(), latLonToString()
cogo.ts	radiation(), bearingIntersection(), tienstraResection(), distanceIntersection()
geometry.ts	midpoint(), wcbToQuadrant(), quadrantToWCB(), angularMisclosure(), pointInPolygon(), lineIntersection(), circularBuffer(), offsetPolyline()
curves.ts	curveElements(), curveStakeout()
parser.ts	parseDelimitedFile(), pointsToCSV(), validatePoints()

5.2 Area Methods (Basak Ch. 7)

The engine implements coordinateArea() (Shoelace method). The agent must also support the following field offset methods which are presented separately in the UI:

Method	Formula	Use When
Mid-Ordinate Rule	A = d × (O1+O2+...+On)	Offsets at mid-intervals
Average-Ordinate Rule	A = L/(n+1) × ΣO	Equal intervals, all endpoints
Trapezoidal Rule	A = d×[(O0+On)/2 + ΣOi]	More accurate than avg-ordinate

Simpson's Rule	A = d/3×[(O0+On)+4(odds)+2(evens)]	Best for curved boundaries
Coordinate Method	A = 0.5 × |Σ(EiNi+1 - Ei+1Ni)|	Best when coords are known

5.3 Volume Computation (Basak Ch. 8)

- Average End Area: V = L/2 × (A1 + A2)
- Prismoidal Correction: Cp = L/6 × (h1 - h2)^2 × correction factor
- Always apply prismoidal correction unless user specifies otherwise
- Always separate and report: CUT volume, FILL volume, NET volume

5.4 Traverse Standards

Survey Type	Required Precision	Grade
Urban cadastral	1 : 5 000 or better	excellent
Suburban survey	1 : 3 000 or better	good
Rural survey	1 : 1 000 or better	acceptable
Rough survey	< 1 : 1 000	poor — WARN USER

5.5 Leveling Allowable Misclosure

Class	Allowable Misclosure	Notes
Precise leveling	±6√K mm	K = route length in km
Ordinary leveling	±12√K mm	Standard engineering surveys
Rough leveling	±24√K mm	Reconnaissance only

5.6 Angular Misclosure Tolerances

Class	Tolerance	Notes
Precise work	±10" × √n	n = number of stations
Ordinary work	±60" × √n	Standard traverse
Rough work	±2' × √n	Reconnaissance

5.7 Coordinate Conversion

- WGS84 ellipsoid: a = 6,378,137.0 m, f = 1/298.257223563
- UTM scale factor k0 = 0.9996, False easting = 500,000 m
- Southern hemisphere: False northing = 10,000,000 m
- Round-trip accuracy: better than 1 mm
- Always state UTM zone and hemisphere in output

5.8 Horizontal Curves (Basak Ch. 10)

All five curve types must be supported:

Curve Type	Key Parameters	Notes
Simple Circular	T, L, C, E, M, D, PC, PT chainages	Most common
Compound Curve	Two arcs, sub-tangents, common tangent	Road intersections
Reverse Curve	Two opposite arcs — warn: hazardous at speed	Low-speed roads only
Transition (Clothoid)	$L = v^3/CR$, shift $S = L^2/24R$	Roads, railways
Vertical Curve	Parabolic: $y = (g2-g1)x^2/2L$	Road profile design

6. API CONTRACT

6.1 Endpoint Specification

All endpoints use JSON. All inputs validated with Zod. All calculation results include full working (inputs, formula, result, check).

Endpoint	Input	Output
POST /api/cogo/distance-bearing	{ x1,y1,x2,y2 }	DistanceBearingResult + full working
POST /api/cogo/polar-point	{ from, bearingDeg, distance }	Computed Point2D
POST /api/cogo/intersection	{ stationA, bearingA, stationB, bearingB }	IntersectionResult
POST /api/cogo/resection	{ p1,p2,p3, alpha, beta }	ResectionResult
POST /api/traverse/bowditch	{ stations: NamedPoint2D[] }	TraverseResult
POST /api/traverse/transit	{ stations: NamedPoint2D[] }	TraverseResult
POST /api/leveling/rise-fall	{ readings, openingRL, closingRL? }	LevelingResult
POST /api/leveling/hoc	{ readings, openingRL, closingRL? }	LevelingResult
POST /api/area/coordinate	{ points: Point2D[] }	AreaResult
POST /api/area/offset	{ method, ordinates, interval }	AreaResult
POST /api/curves/elements	{ radius, deflectionDeg }	CurveElements
POST /api/curves/stakeout	{ pi, bearing, radius, delta, chainagePI, interval }	CurveStakeout
POST /api/coords/utm-to-geo	{ easting, northing, zone, hemisphere }	LatLon
POST /api/coords/geo-to-utm	{ latitude, longitude, zone? }	UTMCoord
POST /api/projects	{ name, location, utm_zone, hemisphere }	Project
POST /api/projects/:id/points	{ points: NamedPoint3D[] }	Inserted point IDs
POST /api/import/csv	{ file: FormData }	ParsedSurveyFile
POST /api/reports/generate	{ project_id, include: [] }	PDF blob

6.2 Standard Response Envelope

```
{ "ok": true, "value": { ... }, "working": { "formula": "...", "substituted": "...",
"check": "PASS" } }
{ "ok": false, "error": "Traverse requires at least 3 stations." }
```

! *Never return a bare number. All responses include the full working object so the frontend can display the audit trail.*

7. FRONTEND & UI SPECIFICATION

7.1 Page Structure

Route	Page	Description
/	Landing page	CTA: Start Calculating (no login). Feature overview.
/tools	Quick Tools hub	Grid of 12 calculator cards. No project needed.
/tools/distance	Distance & Bearing	Two-point input, instant results, bearing in DMS
/tools/bearing	Bearing Calculator	WCB + Quadrant bearing output
/tools/area	Area Calculator	Coordinate method + offset method toggle
/tools/traverse	Traverse Adjustment	Point table + Bowditch/Transit toggle
/tools/leveling	Leveling	Level book table + Rise&Fall / HoC toggle
/tools/coordinates	Coordinate Conversion	UTM ↔ Lat/Lon + DMS ↔ Decimal
/tools/curves	Horizontal Curves	Curve elements + stakeout table
/tools/cogo	COGO Tools	Radiation, Intersection, Resection
/dashboard	Project Dashboard	Project list, New Project button
/project/:id	Project Workspace	Map + tools panel + coordinate table
/project/:id/report	Report Generator	Checkboxes, Generate PDF

7.2 Design System

Element	Specification
Theme	Dark industrial — like the inside of a Leica total station. Black backgrounds, amber (#E8841A) accents.
Typography	Share Tech Mono for all numbers and coordinates. Barlow Condensed for labels and headings.
Numbers	Always 4 decimal places for coordinates, DDD° MM' SS.SSS" for bearings.
Colours	Blue #1A3A5C (primary), Amber #E8841A (accent/action), Green #4ADE80 (pass/ok), Red #EF4444 (error), Yellow #FCD34D (warning).
Tables	TanStack Table. Inline editing. Monospace numbers. Import CSV button on every table.

Map	Leaflet. Dark tile layer. Points as orange hexagon markers. Traverse lines in blue. Parcels filled in semi-transparent blue.

7.3 Calculator UI Pattern

Every calculator follows this exact layout:

1. Title bar with module name and a tooltip icon (?) explaining what the module does
2. Input section: labelled fields, units shown next to every field, monospace font for numeric inputs
3. Calculate button — triggers calculation
4. Result section: appears below, shows all computed values with labels and units
5. Working section (collapsible): shows formula → substitution → result → arithmetic check
6. Action buttons: Copy Result, Add to Project, Export CSV

7.4 Map Interactions

- Click on map: creates a survey point at cursor coordinates
- Click two existing points: automatically computes distance and bearing
- Draw polygon (click + close): automatically computes area
- Drag point: updates coordinates in table and recomputes all dependent calculations
- Coordinate display: live E/N at cursor bottom-left, updates as mouse moves
- Layers toggle: Points / Traverse Lines / Parcels / Satellite / OpenStreetMap

7.5 Mobile Layout

- Full-screen map view
- Tools panel slides up from bottom (50% screen height)
- Coordinate table collapses to a summary bar, expands on tap
- All tap targets minimum 44×44 px

8. AGENT CALCULATION BEHAVIOUR

8.1 Output Format Rules

- Coordinates: ALWAYS 4 decimal places (e.g. 345234.2230, not 345234.22 or 345234.223)
- Bearings: ALWAYS DDD° MM' SS.SSS" (e.g. 045° 30' 22.450", never 45.5°)
- Also output Quadrant bearing alongside WCB for any bearing result
- Areas: m² (4dp), hectares (6dp), acres (4dp) — always all three
- Distances: metres to 4dp
- Staff readings and RLs: metres to 3dp
- Never round intermediate calculations — only round final outputs

8.2 Mandatory Working Display

Every calculation must display these five sections:

Section	Content
Given:	List all input values with units
To find:	State what is being computed
Solution:	Numbered steps: formula name → symbolic formula → substituted values → computed result
Check:	Arithmetic verification — state PASS ✓ or FAIL ✗
Result:	Boxed or highlighted final answer with all unit conversions

8.3 Traverse Table Format

Every traverse output must include a complete Gale's Table:

Line	Dist (m)	Bearing	Lat (+N)	Lat (-S)	Dep (+E)	Dep (-W)	Adj Lat	Adj Dep

8.4 Level Book Format

Station	BS	IS	FS	Rise	Fall	RL (m)	Remarks

9. FIELD DATA IMPORT

9.1 Supported Formats

Format	Handling
CSV with header	POINT,EASTING,NORTHING,ELEVATION (auto-detected)
CSV without header	Columns detected by position and value range
Tab-delimited TXT	Total station export — tab separated, with or without header
Space-delimited DAT	Field controller format — multiple spaces as delimiter
Leica GSI-8	Word-based: ID 11=point, 81=easting, 82=northing, 83=elevation

9.2 Import Workflow

7. User selects file — parser auto-detects format and delimiter
8. Data displayed in preview table for user confirmation
9. Validation runs: duplicate names flagged, coordinate range checked, elevation sanity checked
10. Warnings shown per-row — NO rows silently dropped
11. User confirms: 'Import N points to project'
12. Points saved to Supabase, PostGIS geometry column populated
13. Leaflet map updates — points plotted immediately
14. If >= 3 points: area automatically computed and displayed
15. If points form a traverse sequence: bearings and distances computed for all legs

> **!** *The agent must NEVER silently drop invalid rows. Every skipped row generates a named warning: 'Row 4 (point P3): easting value "abc" could not be parsed — skipped.'*

9.3 Automatic Post-Import Calculations

- All inter-point bearings and distances computed and stored in traverse_legs
- Polygon area computed if >= 3 points and points form a closed boundary
- Coordinate range check: easting 100,000–900,000 for UTM
- Duplicate name check across all points in project

10. OUTPUT & REPORT STANDARDS

10.1 PDF Report Contents

The Generate Report page presents these sections as checkboxes. All checked sections are included:

Section	Contents
Cover Page	Project name, client, location, surveyor name, date, UTM zone, datum
Coordinate Table	All survey points: name, easting (4dp), northing (4dp), elevation (3dp)
Traverse Adjustment	Full Gale's Table, closing errors, precision ratio, method used
Leveling Summary	Full level book, arithmetic check result, misclosure, adjusted RLs
Area Computation	Method used, full working, result in m²/ha/acres
Curve Data	All curve elements, stakeout table with deflection angles
COGO Results	All coordinate geometry results computed in the project
Closure Report	Summary of all misclosures — PASS/FAIL against tolerances

10.2 Precision Standards Summary

Value Type	Storage	Display
Coordinates stored	NUMERIC(14,4)	0.1 mm — 4 decimal places
Coordinates displayed	0.0001 m	4 decimal places
Bearings stored	NUMERIC(10,6) decimal degrees	Full precision
Bearings displayed	DDD° MM' SS.SSS"	3dp seconds
Areas displayed	0.0001 m², 0.000001 ha	4dp and 6dp
RLs displayed	0.001 m	3 decimal places
Intermediate values	Full floating point	Never rounded mid-calculation

11. BUILD PHASES & MILESTONES

Phase 1 — Core Engine + Quick Tools (Weeks 1–4)

The engine is already built (70 tests passing). Phase 1 is wiring it to the UI.

- Next.js project setup with Tailwind and shadcn/ui
- /tools/distance — distance + bearing calculator, full DMS output
- /tools/bearing — WCB + quadrant bearing
- /tools/area — coordinate method + Simpson's rule for offsets
- /tools/coordinates — UTM ↔ Lat/Lon, DMS ↔ Decimal
- /tools/leveling — Rise & Fall + Height of Collimation, full level book
- /tools/traverse — Bowditch + Transit, full Gale's Table
- CSV upload → parse → preview → coordinate table (no Supabase yet)

Phase 1 Exit Criteria

- All 7 quick tools produce correct results verified against Basak worked examples
- Traverse arithmetic check passes on test data
- Level book arithmetic check passes on test data

Phase 2 — Map + Projects (Weeks 5–8)

- Supabase setup: all schema tables created
- Auth: email/password login via Supabase Auth
- /dashboard: project list, New Project, open existing
- /project/:id: Leaflet map with point plotting
- CSV import → Supabase → map display
- Click map → create point → table updates
- Draw polygon → compute area → display
- Basic PDF report generation (coordinate table + traverse table)

Phase 3 — Curves + Full Reports (Weeks 9–12)

- /tools/curves: simple and compound curves, stakeout tables
- Transition and vertical curves
- Full PDF report with all 8 sections
- DXF export (AutoCAD compatible)
- Volume computation (cross-sections, cut/fill)

Phase 4 — Advanced (Post-Launch)

- Least Squares network adjustment (NumPy via Python API)
- Leica GSI file parser
- Drone orthophoto overlay
- Multi-user collaboration
- Public API with API keys for third-party integrations

12. ERROR DETECTION RULES

The agent must automatically detect and flag these conditions. Do not wait for the user to ask.

Condition	Detection Method	Agent Action
Traverse precision < 1:1000	After Bowditch/Transit	POOR PRECISION WARNING — show 1:N, flag in red
Level misclosure > allowable	After RL computation	UNACCEPTABLE MISCLOSURE — do not adjust, show limit
Angular misclosure > ±60"√n	After summing angles	Flag, show theoretical vs observed sum
Easting outside 100K–900K m	On every coordinate input	COORDINATE RANGE WARNING — check for E/N swap
Duplicate point names	On CSV import	List all duplicates by row number, require resolution
All bearings ~180° off	Statistical check on traverse	Suggest: bearings may be measured from South
Staff readings inconsistent scale	Range check on leveling	Suggest: possible feet/metres mismatch
Curve radius < minimum for design speed	After curve elements	DESIGN STANDARD VIOLATION — show minimum radius
Negative shoelace area	After coordinateArea()	Clockwise winding — take absolute value, note it
Simpson's rule with odd intervals	Before area calc	Auto-apply Simpson's + Trapezoid for last strip
Closing point ≠ opening point in closed traverse	Always computed	Show linear misclosure even if user didn't ask
Near-zero denominator in resection	Tienstra formula	DANGER CIRCLE WARNING — instrument near control circle

13. GLOSSARY

Term	Definition
Azimuth / WCB	Whole-Circle Bearing: 0°–360°, measured clockwise from North. What total stations display.
Quadrant Bearing	N 45° E style notation used in deed descriptions and legal surveys.
Latitude (traverse)	North/South component of a traverse leg. L = d·cos(θ). Not geographic latitude.
Departure (traverse)	East/West component. D = d·sin(θ).
Bowditch Rule	Traverse adjustment proportional to leg length. Use when angular ≈ linear accuracy.
Transit Rule	Adjustment proportional to |latitude| and |departure|. Use when angular > linear.
Reduced Level (RL)	Elevation above datum, computed by leveling. Also called 'height'.
Misclosure	The gap between where a closed traverse or leveling run ends and where it should. Not 'error'.
Chainage	Distance measured along a road/railway/pipeline from a reference point.
PC	Point of Curvature — start of a horizontal curve.
PI	Point of Intersection — where the two tangents meet.
PT	Point of Tangency — end of a horizontal curve.
COGO	Coordinate Geometry — computing positions from angles and distances.
Resection	Finding the instrument's position from observations to known points.
Radiation	Computing a new point from the instrument using bearing and distance.
Tienstra	Three-point resection formula. Instrument's position from 2 angles to 3 known points.
PostGIS	PostgreSQL extension for spatial data. Stores geometry columns, runs spatial queries.
UTM	Universal Transverse Mercator. A projected coordinate system dividing the world into 60 zones.
DMS	Degrees Minutes Seconds — e.g. 45° 30' 22.450". Standard bearing notation.